

DIGITALAIKART · PREMIUM DIGITAL PRODUCT

The GenAI Engineer Career Kit

2026-27 · India Edition

From any tech job to a paid AI engineering role — the skill stack employers actually hire for, four portfolio blueprints, a 45-question interview bank and a week-by-week 90-day plan.

Faisal Fayaz Lone · First Edition · October 2026

How to Use This Kit

This is a working career kit, not a textbook. It is built for one specific person: an Indian software professional, analyst, tester, support engineer or recent graduate who can already write some code and wants to move into a paid AI engineering role within the next three to six months — without a Master's degree, a PhD or a Rs 5-lakh bootcamp.

Every number, tool name and price in this kit was verified in October 2026. AI moves fast, so the market figures and model prices are dated — re-check them when you apply. The methods and the interview patterns will stay useful far longer.

What is inside

- **Part 1 — The 2026 market:** what is actually hiring in India, the salary bands, and why experience now beats enthusiasm.
- **Part 2 — The role map:** AI Engineer vs ML Engineer vs GenAI Engineer vs MLOps — and which to target.
- **Part 3 — The skill stack:** the seven skills that get hired, with the proof you must show for each.
- **Part 4 — The 90-day roadmap:** a week-by-week plan you can run around a full-time job.
- **Parts 5-8 — Four portfolio blueprints:** a production RAG service, a tool-using agent with guardrails, an evaluation harness and an MCP server.
- **Part 9 — The tool and model landscape:** the current models, prices, vector databases and frameworks, dated October 2026.
- **Part 10 — The interview bank:** 45 questions with senior-level answers, including the RAG system-design round.
- **Parts 11-13 — Getting hired:** resume and LinkedIn rewrite, salary negotiation in India, and a 30-day action plan.

How to read it

1. Read Parts 1-4 in one sitting. They give you the map and the plan.
2. Pick **two** portfolio blueprints from Parts 5-8 and build them for real. Two deep projects beat four shallow ones.
3. Study the interview bank in Part 10 out loud, not silently. Interview answers are a performance, not a memory test.
4. Run the 30-day action plan in Part 13 and start applying before you feel “ready”.

A fair promise

This kit cannot guarantee a job — no book can, and anyone who promises that is selling you something. What it does give you is the same skill order, the same portfolio bar and the same interview answers that working AI engineers used to switch roles in 2026. The rest is your reps.

Educational guidance only. This kit is not legal, financial or immigration advice, and salary figures are market ranges, not offers. © 2026 Digitalaikart. Personal use; please do not redistribute.

PART 1 The 2026 AI Job Market in India

Before you spend ninety days learning anything, you should know exactly what the market is paying for right now. The picture in October 2026 is unusually clear — and unusually favourable if you already know how to build software.

The headline numbers

- India had **1.23 lakh active technology job openings** in October 2026 — the highest level in 18 months, up about 17% from a year earlier.
- But overall white-collar hiring grew only **1% year-on-year** in September 2026, while **AI and machine-learning hiring jumped 20%**. AI is growing twenty times faster than the rest of the market.
- Demand for **agentic AI engineers** rose **260% year-on-year** in 2026. GenAI solutions architects and AI product owners each grew around 120%, LLM engineers 86.5% and MLOps engineers 82.2%.
- Demand for GenAI/LLM skills has surged roughly **300% since 2024**, and there is about **one qualified engineer for every ten open GenAI roles**.
- India faces a projected **53% AI talent gap** in 2026 — the fastest-growing roles are also the least saturated.
- Global Capability Centres are projected to hire about **5.1 lakh people in 2026**, and 64% of their new roles require AI, data-science or intelligent-automation skills.

The one-sentence summary

The market is **selective but real**: it is not hiring more people in general, it is hiring far more people who can build with AI. That is exactly the gap this kit closes.

Where the demand actually concentrates

Not all AI skills are equal. NASSCOM's 2026 talent analysis measured how often each skill appears in job posts relative to the number of professionals claiming it. Applied skills sit at the top; theoretical deep learning sits at the bottom, drowning in candidates.

Skill	Job posts per 1,000 professionals	Signal
Java	57	Strong demand
Analytical skills	44	Strong demand
AWS / cloud	42	Strong demand
AI (general / applied)	36	Strong and rising
SQL	32	Solid
Python	26	Solid, high volume
Data engineering (Kafka, Spark)	17	Moderate
Machine learning	10	More profiles than jobs
Generative AI	7	Growing fast — becoming a baseline

Skill	Job posts per 1,000 professionals	Signal
Deep learning	3	Major mismatch — 1.1M profiles, 3,600 posts

Read that table carefully. Employers are not asking for people who can explain backpropagation. They are asking for people who can put a language model inside a real product, on real data, with cloud and SQL and engineering fundamentals underneath it. That is the profile this kit builds.

The salary bands you can target

These are 2026 India market ranges for AI engineering roles, blended across product companies, GCCs and funded startups. A GCC or a product company pays at the top of the band; a services firm at the bottom. Treat them as the shape of the ladder, not a promise.

Role / skill	Fresher-early (₹ LPA)	Senior (₹ LPA)
GenAI / LLM engineering	8-15	40-80+
Agentic AI engineering	10-18	30-50+
MLOps / LLMOps	6-10	58-60
Data engineering	6-14	25-45
Cloud / platform engineering	6-12	25-48
Cybersecurity / cloud security	6-12	25-50+

What the recovery rewards

The hiring recovery is being driven by **experienced** talent — mid-senior roles were 55% of all tech openings in October 2026. If you already have one to eight years of software or analytics experience, that experience is not a liability in an AI interview. It is your single biggest advantage over candidates who learned ML theory but never shipped a service. Lead with it.

PART 2 The Role Map — Pick Your Target

“AI job” means at least five different jobs. Applying to all of them at once is why most people get silence. Choose one target, build the proof that target needs, and aim everything at it.

Role	What you actually do	Core skills	Best entry for you
AI Engineer (applied / product)	Build LLM features inside a product: chat, search, agents, extraction.	Python/TS, LLM APIs, RAG, evals, FastAPI, cloud	Software engineer, 1–8 yrs
GenAI / LLM Engineer	Own the model layer: prompting, RAG quality, agents, guardrails, cost.	LLM APIs, RAG, agents, eval, fine-tuning basics	Backend / full-stack engineer
ML Engineer	Train, fine-tune and serve models; own model quality metrics.	Python, PyTorch, training, LoRA, serving	Engineer with maths appetite
MLOps / LLMOps Engineer	Deploy, monitor and scale models and pipelines in production.	Docker, K8s, CI/CD, vLLM, observability	DevOps / backend engineer
Data Engineer	Build the pipelines every AI system depends on.	SQL, Spark, dbt, Airflow, Kafka	Analyst / backend engineer
Data Scientist / Analyst	Turn data into decisions; add AI to reporting.	SQL, Python, stats, viz, some LLM	Analyst, fresher

The fastest credible path in 2026

For someone who can already write software, the highest-probability landing spot is **AI product engineering** — an AI Engineer or GenAI Engineer role at a company building AI-first products, a GCC, or a funded startup. The reason is simple: these roles want “strong software engineering fundamentals” plus AI skills. Most candidates bring one or the other. You bring both.

- **Target AI product engineering** if you can code and want the biggest volume of openings. This is the default recommendation of this kit.
- **Target MLOps** if you come from DevOps, SRE or backend and enjoy reliability more than research. It is one of the hardest roles to fill and pays accordingly.
- **Target data engineering** if you are strong in SQL and want the steadiest, least-hyped demand — it sits underneath every AI programme.
- **Do not target pure ML research** unless you have publications. It is a tiny share of openings and the most competitive.

AI Engineer vs ML Engineer — the difference interviewers expect you to know

An **ML Engineer** trains and tunes models — data, features, training runs, evaluation of model quality. An **AI Engineer** builds systems around foundation models that someone else trained: retrieval, prompting, tool calling, guardrails, latency and cost. In 2026 the AI Engineer role is the broader and faster-growing one, and it is the role this kit prepares you for. Say this distinction clearly in interviews — it signals you understand the market, not just the tools.

PART 3 The Seven Skills That Get Hired

Seven skills, in the order you should learn them. For each one, the right-hand column is the “proof” — the artefact that convinces a hiring manager you actually have the skill. A skill without proof does not get hired.

#	Skill	What “good” looks like	Your proof
1	LLM API fluency	Streaming, structured outputs, retries, error handling, token accounting	A small CLI or web app calling a real API
2	Prompting as engineering	System prompts, few-shot, function/tool calling, evaluation of prompt changes	Versioned prompts with before/after scores
3	RAG	Chunking, embeddings, hybrid search, reranking, citations, retrieval metrics	A document Q&A; service with cited answers
4	Agents & MCP	The tool-calling loop, memory, scoped tools, human approval	An agent that automates a real task, safely
5	Evaluation	Golden sets, LLM-as-judge, regression gates in CI	An eval harness that caught a regression
6	Fine-tuning (basics)	LoRA/QLoRA, when not to fine-tune, before/after metrics	A tuned small model vs a prompt baseline
7	Serving & MLOps	vLLM, batching, caching, observability, cost per query	A deployed service with cost and latency numbers

1. LLM API fluency

You must be able to call a model API without a framework and understand what you are sending. That means streaming responses, asking for structured JSON output, handling rate limits and timeouts, and knowing that input and output tokens are billed separately. This is the foundation everything else sits on — do not skip it to reach RAG faster.

2. Prompting as engineering

Prompting in 2026 is not “vibes”. It is designing a system prompt, giving few-shot examples, and using the model's native tool-calling to return validated structured output. The senior signal is that you test prompt changes against a scored set instead of eyeballing a single example.

3. RAG — retrieval-augmented generation

RAG is the single most in-demand AI engineering pattern in India right now, and the most tested topic in interviews. It is the architecture that lets a model answer from your documents instead of its training data. The parts that matter: structure-aware chunking, embedding model choice, a vector index, hybrid search that combines keyword (BM25) and vector results, a cross-encoder reranker, grounded generation with citations, and — the part beginners skip — retrieval evaluation. If retrieval returns the wrong chunks, no prompt can save the answer.

4. Agents and MCP

An agent is a model in a loop that can call tools, observe results and decide the next step. In 2026 “I built an agent” impresses nobody; “I bounded the agent with scoped tools, a step budget and human approval for destructive actions” does. MCP (the Model Context Protocol) is the emerging standard for connecting models to tools and data sources, and shipping one MCP server is a strong, timely

portfolio signal.

5. Evaluation — the real differentiator

“It looks good” does not ship. Teams hire people who can build a golden dataset, score outputs with a mix of deterministic checks and an LLM judge calibrated against human labels, and gate prompt or model changes on the result in CI. Very few junior candidates have shipped real evals. If you learn only one thing beyond RAG, learn this.

6. Fine-tuning — and when not to

LoRA and QLoRA are the standard ways to adapt a small open model. The interview trap is reaching for fine-tuning when RAG or a better prompt would do. The rule: prompt first, RAG for knowledge, fine-tune last — and only for behaviour, format or cost, never as a way to inject facts. Know before-and-after metrics for the one narrow task you tuned.

7. Serving and MLOps

The last mile: serving models efficiently (vLLM, continuous batching, KV-cache management, quantisation), caching repeated work, tracing every call, and knowing your cost per query and p95 latency. This is where an experienced engineer separates from a tutorial follower, because you already think in terms of reliability, budgets and failure modes.

Breadth beats depth here — with one exception

For an entry AI engineering role you need working competence across all seven, not mastery of one. The exception is evaluation: go deep there, because it is the skill fewest candidates can demonstrate and the one that most changes how a hiring manager reads your GitHub.

PART 4 The 90-Day Roadmap

Budget 8–10 focused hours a week — evenings and weekends are enough. The plan assumes you can already write basic code. Each week ends in an artefact you keep, not a video you watched. Reserve the last two weeks for packaging and interview practice, not new learning.

Week	Focus	Artefact you finish with
1	Python refresh, Git, environments; call a model API directly	A working CLI script that streams a response
2	Structured outputs, tool calling, error handling, token accounting	A small app returning validated JSON
3	RAG part 1 — ingestion, chunking, embeddings, a vector index	A pipeline that indexes a real document set
4	RAG part 2 — hybrid search, reranking, cited generation	Grounded answers with source citations
5	RAG evaluation — golden set, recall@k, faithfulness	A retrieval + faithfulness score, committed
6	Production polish — FastAPI, tracing, caching, cost per query	Portfolio Project 1 complete and deployed
7	Agents — the tool-calling loop without a framework	An agent completing a real multi-step task
8	Guardrails — scoped tools, allowlists, human approval, injection tests	An agent you would trust with a write action
9	Agent evaluation — task-success sets, failure traces	Portfolio Project 2 complete
10	The eval harness — extract scoring into a reusable tool + CI	Portfolio Project 3 — your strongest signal
11	MCP server + README pass on everything (diagrams, metrics, trade-offs)	Portfolio Project 4 and a clean GitHub
12	Interview mode — two rehearsed project stories, one mock design round	Applications out, interviews booked

The weekly scorecard

Track five numbers every week. They tell you whether you are actually moving or just watching tutorials.

- **Hours learned** — aim 8–10. Below 5, your timeline slips.
- **Project commits** — aim at least 3 public commits a week. An empty GitHub is the most common reason for silence.
- **Problems solved** — the interview questions you answered out loud this week.
- **Written notes** — one short note on a failure you debugged. This becomes blog and interview material.
- **Applications sent** — start in week 9, not week 12. Apply before you feel ready.

The bottleneck is never the learning

Almost everyone who fails to switch roles does not fail because the material was too hard. They fail because they never **shipped the projects in public**. Two projects with real metrics in the README will beat a stack of certificates every time.

PART 5-8 Four Portfolio Blueprints

The 2026 market rewards demonstrated AI-shipping far more than credentials. These four projects are written to the bar that actually moves a reviewer — each ends in a public repo with a real metric, an architecture diagram and an honest trade-offs section. Build two of them properly. The third and fourth are what separate you from the crowd.

The rule that decides everything

A recruiter should be able to open your repo and see a **working system**, a **number** that proves it works, and a **diagram** that explains it — within five minutes, without running anything. Optimise your README for that five minutes.

Project 1 — A production RAG service

The problem it solves. Enterprise knowledge is locked in documents a model was never trained on. RAG lets you answer questions from that knowledge, with citations, without retraining anything.

What you build.

- An ingestion pipeline that parses a real document set, chunks it structure-aware, embeds it and stores it in a vector index with metadata.
- Hybrid retrieval that fuses dense vector search with BM25 keyword search, then reranks the top candidates with a cross-encoder.
- Grounded generation that answers only from retrieved passages, cites each source, and **abstains** when retrieval comes up empty.
- A retrieval and faithfulness evaluation that proves the system works — and that you can re-run after any change.

Suggested stack. Python, FastAPI, one LLM API (e.g. a frontier model for generation, a cheaper one for query rewriting), an embedding model, a vector store (pgvector, Qdrant or Pinecone), a reranker, LangSmith or Langfuse for tracing.

Your README must show.

- A golden set of 50+ questions with the correct source chunks labelled, committed to the repo.
- Reported numbers: recall@5 (did the right chunk get retrieved?) and faithfulness (is every claim supported by a retrieved passage?).
- An architecture diagram and a trade-offs section: why this chunk size, this embedding model, this index.
- A measured **cost per query** and a **p95 latency** number.
- A live demo link or a recorded walkthrough.

Common mistakes.

- Using a toy document set. Use a real corpus you care about — your team's handbook, a public standard, a niche domain you know deeply.
- Shipping a chatbot demo with no evaluation. Without recall and faithfulness numbers it reads as a tutorial.
- Retrieving 20 chunks and flooding the context window. Cap and rerank to the best 5–8.
- Skipping abstention. A system that says “I could not find this” is more trustworthy than one that guesses.

Project 2 — A tool-using agent with guardrails

The problem it solves. Many real tasks are multi-step: research across sources, triage an issue, draft a report. An agent plans, calls tools and iterates — but only if it is bounded so it cannot do damage.

What you build.

- An agent that completes one genuine multi-step task with two or three tools (for example: triaging GitHub issues, or researching a topic across several sources).
- Scoped tool permissions: an allowlist for any write action and human approval before anything destructive.
- Failure recovery: the agent detects a tool error, retries with backoff, or degrades gracefully.
- Prompt-injection tests proving that content retrieved from a tool cannot hijack the agent's instructions.

Suggested stack. Python; an agent framework (LangGraph, or a hand-written tool-calling loop); the model's native function calling; a small vector store if the task needs memory; tracing.

Your README must show.

- A trace showing the agent recovering from a tool error — not just the happy path.
- A task-success evaluation: run the agent on 20–50 real tasks and report the success rate.
- An honest limitations section: what the agent refuses to do and where it still fails.
- The allowlist and approval gate visible in the code.

Common mistakes.

- Building a demo that only works on the happy path. Interviewers ask about failures; have the trace.
- Giving the agent an unrestricted shell or an unbounded write tool. That is an instant red flag.
- Ignoring prompt injection. Retrieved content is untrusted input — say how you defend against it.

Project 3 — An evaluation harness

The problem it solves. Teams ship AI features faster and safer when quality is measured automatically. Almost no junior candidate has built one — which is exactly why it is your strongest portfolio piece.

What you build.

- A reusable harness that scores an LLM application: a golden dataset, deterministic checks (schema, exact match) plus an LLM judge with a clear rubric.
- Judge calibration: report agreement between the LLM judge and a human-labelled sample, and watch for position and verbosity bias.
- A regression gate wired into CI: a prompt or model change that drops quality fails the build.

- Runs against your RAG project and your agent project, so it proves both.

Suggested stack. Python, pytest, a frontier model as judge, a small human-labelled sample, GitHub Actions for the CI gate.

Your README must show.

- A README note about a regression it actually caught — the story that makes the harness real.
- Judge-human agreement reported as a number.
- The CI workflow committed and green.
- Clear instructions so a reviewer can point it at their own app.

Common mistakes.

- Using the same model to judge its own output with no human calibration. Report the agreement.
- Treating eval as pass/fail. Quality is a distribution — track scores over runs.
- Scoring only the final answer. Score retrieval separately from generation so you know which half to fix.

Project 4 — An MCP server

The problem it solves. MCP (the Model Context Protocol) is the emerging standard for connecting models to tools and data. Shipping one shows you understand the layer everyone is now building on.

What you build.

- Wrap one real API, database or dataset you know well as an MCP server with a handful of well-designed tools.
- Clear tool schemas, good descriptions and sensible error messages — the quality of the tool contract is the whole point.
- A short README showing it working inside an MCP-capable client (a coding assistant or agent).

Suggested stack. The MCP SDK (Python or TypeScript), one real data source, a host client for the demo.

Your README must show.

- A published repo with the server and a working demo.
- A short post or README section on the tool-design decisions you made.
- Evidence it is used by at least one real client.

Common mistakes.

- Wrapping a toy API. Wrap something you genuinely use.
- Vague tool descriptions. The model can only use a tool as well as its description explains it.
- Ignoring auth and scoping. Expose only what the task needs.

Packaging beats building

Same projects, two candidates: one with a bare README and no numbers, one with an architecture diagram, a metrics table, a cost-per-query figure and a trade-offs section. The second candidate gets the interview every time. Spend week 11 polishing presentation — it is not optional.

PART 9 The Tool and Model Landscape

Verified against official sources in October 2026. AI pricing and model names change every few weeks — always confirm the current price on the provider's own page before you commit to volume. What stays true is the shape: every major vendor sells a flagship, a workhorse and a cheap budget model, and production systems route each request to the cheapest model that can handle it.

Model API prices (USD per 1 million tokens, October 2026)

Model	Tier	Input	Output	Context
GPT-6 Astra (OpenAI)	Flagship	\$10.00	\$50.00	1.05M
Claude Fable 5.1 (Anthropic)	Flagship	\$10.00	\$50.00	1M
Claude Opus 5.5 (Anthropic)	Flagship value	\$4.00	\$20.00	1M
GPT-6.1 Sol (OpenAI)	Workhorse	\$2.00	\$10.00	1.05M
Claude Sonnet 5.5 (Anthropic)	Workhorse	\$2.00	\$10.00	1M
Gemini 3.1 Pro (Google)	Workhorse	\$2.00	\$12.00	1M
Gemini 3.8 Flash (Google)	Budget*	\$0.75	\$3.75	1M
GPT-6 Luna (OpenAI)	Budget	\$0.10	\$0.50	1.05M
Claude Haiku 5.5 (Anthropic)	Budget	\$0.10	\$0.50	1M
text-embedding-3-large (OpenAI)	Embedding	\$0.13	—	—
text-embedding-3-small (OpenAI)	Embedding	\$0.02	—	—

*Gemini 3.8 Flash is at a promotional rate through 31 December 2026; it is scheduled to rise. Cache reads are billed far below input (typically 10% on OpenAI and Anthropic models), which is what makes agent workloads affordable. Batch APIs are usually half price.

The rest of the stack

Layer	What is used in production in 2026
LLM providers	OpenAI (GPT-6 family), Anthropic (Claude 5 family), Google (Gemini 3.x), plus open-weight models (Llama, Qwen, DeepSeek, Mistral) for self-hosting
Vector stores	pgvector (Postgres), Qdrant, Pinecone, Weaviate, Chroma for local dev
Orchestration	LangChain, LlamaIndex, LangGraph (agents), DSPy, PydanticAI, and thin hand-written layers
Rerankers	Cohere Rerank, open BGE cross-encoder models
Evaluation	RAGAS for RAG metrics, custom LLM-as-judge harnesses, Braintrust
Observability	LangSmith, Langfuse — tracing every LLM call
Serving / inference	vLLM and TGI for self-hosted; continuous batching, KV-cache management, quantisation
Integration standard	MCP (Model Context Protocol) for connecting models to tools and data

Cost reality — the numbers interviewers want

A 3:1 blended rate (input : output) is a useful rough rule. At 100M input and 20M output tokens a month, a flagship tier costs around \$2,000/month, a workhorse tier around \$400, and a budget tier around \$20 — a 100x spread. Model choice dominates your bill, so the senior move is always

routing: send simple queries to a cheap model, reserve the flagship for the hard 10%.

PART 10 The Interview Bank — 45 Questions

These are the questions that actually come up in GenAI, RAG and AI-engineer loops in 2026, with the answers that separate a senior candidate from a tutorial follower. Practise them **out loud**. The interviewer is testing whether you think in trade-offs and have dealt with production reality — not whether you can recite a definition.

A. LLM fundamentals and prompting

Q. What is an AI engineer, and how is it different from an ML engineer?

An ML engineer trains and tunes models — data, features, training runs, model quality. An AI engineer builds systems around foundation models someone else trained: retrieval, prompting, tool calling, guardrails, latency and cost. In 2026 the AI engineer role is the broader and faster-growing one, and it is mostly a software-engineering job with LLM depth.

Q. At a high level, what happens when you send a prompt to an LLM?

Your text is tokenised into pieces, embedded through the network's layers to produce a probability distribution over the next token, one token is sampled, and the process repeats with the growing sequence until a stop condition. It is autoregressive next-token prediction over a fixed context window — there is no lookup of “facts”, which is exactly why grounding with retrieval matters.

Q. What is a token, and why does it matter commercially?

A token is roughly three-quarters of an English word. Providers bill per token, and output tokens usually cost several times more than input tokens. So every design choice — a shorter system prompt, trimming retrieved context, capping output length — is a cost decision. Output is the expensive half, so shape it deliberately.

Q. What is temperature, and when do you lower it?

Temperature controls randomness in sampling. For factual, extraction or coding tasks you want it low (near 0) for consistency. For brainstorming and creative drafting you can raise it. Lowering temperature reduces variance but does not eliminate hallucination — only grounding and evaluation do that.

Q. What is a context window, and what is ‘lost in the middle’?

The context window is the maximum tokens the model can consider at once, including your prompt and its own output. ‘Lost in the middle’ is the measured tendency for models to attend best to the beginning and end of a long context and less to the middle. That is why context ordering matters in RAG, and why retrieving fewer, better chunks beats stuffing the window.

Q. How do you get reliable structured output from a model?

Use the provider's native structured-output or JSON-schema mode rather than asking politely for JSON, and validate the result against a schema (Pydantic or Zod) before you use it. Add a repair path: if validation fails, feed the error back once. Never parse free text with regex in production.

Q. What is tool (function) calling?

You describe available tools with a schema; the model decides when to call one and returns the arguments, your code executes the tool, and the result goes back into the conversation. The model never runs the tool itself — your code does, which is where you enforce permissions and safety.

Q. What is prompt caching and how does it cut cost?

Providers cache the processed prefix of a prompt (the stable system instructions and documents) and bill cache reads at a fraction of the input rate — typically 10% on OpenAI and Anthropic models. In agent workloads the same context is re-sent every turn, so most input tokens become cheap cache hits. Structure prompts so the stable part comes first.

B. Retrieval-augmented generation (RAG)

Q. What is RAG and why use it?

RAG retrieves relevant documents from a knowledge base at query time and injects them into the model's context, so it answers from your data instead of only its training data. It gives you fresh, private, citable knowledge without retraining — and updates instantly when the documents change.

Q. Walk me through a RAG pipeline end-to-end.

Ingestion: parse documents, chunk them structure-aware, attach metadata, embed the chunks and index the vectors. Query time: embed the question with the same model, run a hybrid (vector + keyword) search, fuse the results, rerank the top candidates with a cross-encoder, assemble the best chunks into a token budget, and generate a grounded, cited answer with a refusal path. Then evaluate retrieval and generation separately.

Q. How do you choose chunk size and overlap?

Start around 300–800 characters (roughly 200–500 tokens) with 10–15% overlap, and split on structural boundaries — headings and paragraphs — rather than fixed lengths. Small chunks retrieve precisely but lose context; large chunks carry context but blur relevance. Tune with an evaluation set; do not guess. Code and tables need special handling (AST-aware or row-wise).

Q. What is an embedding, and how do you choose an embedding model?

An embedding maps text to a vector so that similar meanings are close in space. Choose on multilingual ability, dimensions (higher recall but more storage), latency and cost, and whether you must keep data in-house. Crucially, the same model must be used at index time and query time, and switching models means re-embedding the whole corpus.

Q. Why is hybrid search the standard, and how do you fuse results?

Pure vector search blurs exact identifiers — product names, error codes, proper nouns. Pure keyword (BM25) search misses paraphrases. Combining both covers each other's weakness. The standard fusion is Reciprocal Rank Fusion: fuse on rank, not score, because vector similarities and BM25 scores are not comparable. A chunk ranked highly by either list survives.

Q. What is reranking and why does it help?

A reranker is a cross-encoder that scores the question and each candidate chunk together, which is far more accurate than comparing independent embeddings — but far too slow to run over millions of chunks. So you retrieve cheaply and widely (say the top 50) and rerank expensively and narrowly (to the best 5). It is the most reliable way to raise top-k precision.

Q. How do you choose a vector database, and what is an ANN index?

An approximate-nearest-neighbour (ANN) index lets you search millions of vectors in milliseconds by trading a little recall for speed. HNSW is the common default — fast and accurate, more memory; IVF partitions vectors and probes only some partitions — cheaper at huge scale. Choose a store on scale, filtering and permission support, managed versus self-hosted, and operational fit: pgvector if you already run Postgres, Qdrant or Pinecone for a dedicated service. Always check that metadata filters are applied inside the index, not after retrieval.

Q. What are the failure modes of RAG?

Retrieved context is irrelevant (embedding or chunking problem); chunk boundaries split key information; the right document is retrieved but the wrong chunk; the model ignores the retrieved context and hallucinates; and the prompt is flooded with low-value chunks. The fix order: measure retrieval first (recall@k), then fix chunking, embeddings, hybrid search and reranking before you touch the generation prompt.

Q. How do you evaluate a RAG system?

Separate the two halves because they fail separately. Retrieval: a golden set of questions with the correct chunks labelled, scored with recall@k and a rank-aware metric like MRR or nDCG. Generation: hold retrieval fixed and score faithfulness (is every claim supported by a retrieved passage?) and answer relevance, often with an LLM judge calibrated against a human-labelled sample.

Q. How do you keep the index fresh as documents change?

Run an incremental ingestion path: on create, update or delete, re-parse and re-embed only the affected documents, and upsert with the same document id so the index tracks the source of truth. Track the embedding-model version in index metadata so you can migrate deliberately. Re-index changed documents nightly; re-embed everything only when you switch embedding models.

Q. How do you enforce permissions in RAG?

Filter before ranking, not after. Every chunk carries its allowed principals (users, groups, org units); every query carries the asker's principals; both the vector and keyword search run with that filter so a document the user cannot see is never a candidate, never enters the prompt, and can never leak. Authentication and authorisation happen before retrieval.

Q. RAG vs fine-tuning — when do you reach for each?

RAG for knowledge that is fresh, changing, or needs citations and access control. Fine-tuning for behaviour, format, style, or to shrink a large prompt on a high-volume task — never as a way to inject facts. Prompting first (cheapest), RAG for knowledge, fine-tune last. Most production systems combine them: fine-tune the behaviour, retrieve the facts.

Q. Long-context models can take a million tokens — is RAG dead?

No. Long context is expensive, slow, and still suffers from attention dilution; you cannot put an entire enterprise corpus in every prompt, and you still need access control and citations. RAG is not just 'fit text in the window' — it is a search-and-permissions system. Long context changes the tuning; it does not remove the need.

C. Agents and MCP

Q. What is an AI agent?

A model in a loop that can call tools, observe the results and decide the next step until a task is done. The loop is reason → act → observe. The value is handling multi-step tasks a single prompt cannot; the cost is latency, money and harder debugging.

Q. When should you NOT use an agent?

When a deterministic workflow would do. If the steps are known and fixed, a pipeline is cheaper, faster and testable. The honest senior answer is that many 'agent' problems are really workflows. Use an agent only where the path genuinely cannot be pre-defined.

Q. What is the ReAct pattern?

Reason-and-Act: the model alternates between a reasoning step and an action (a tool call), using the observations to guide the next reasoning step. It is the base pattern behind most tool-using agents and is worth being able to write without a framework.

Q. How do you give an agent memory?

Short-term memory is the conversation context. Long-term memory is external — typically a vector store of past facts or summaries retrieved when relevant. Summarise or truncate old turns to control the context budget, and persist state explicitly rather than hoping the window holds everything.

Q. How do you make an agent safe?

Scope its tools to the minimum, allowlist any write action, require human approval before destructive steps, set a step and cost budget, sandbox execution, and defend against prompt injection in retrieved content. Treat a capable agent as an attack surface, not a helper.

Q. How do you evaluate an agent?

Build a task-success set: 20–50 real tasks with a definition of done, run the agent, and report the success rate. Review the traces of failed runs to find where it went wrong. Evaluate the trajectory, not just the final answer, and state honestly what the agent refuses to do.

Q. What is MCP and why does it matter?

The Model Context Protocol is a standard way to connect models to tools and data sources, so one integration works across many clients. It is becoming the 2026 integration standard. Shipping a well-designed MCP server is one of the most timely portfolio signals you can give.

Q. What is multi-agent orchestration, and when is it worth it?

Splitting a task across specialised agents — a planner, a retriever, an executor, a reviewer — often with a manager coordinating them. It can help with genuinely decomposable tasks, but it multiplies cost, latency and failure modes. Start with one agent; add more only when a single agent demonstrably fails.

D. Evaluation

Q. How do you evaluate an LLM system?

Curate a golden set; pick metrics per task (reference-based checks, an LLM judge with a calibrated rubric, human review); score the old and new version on the set; gate deploys on the result; and add online signals (thumbs, deflection) once live. Evals are your unit tests — without them you are shipping on vibes.

Q. What is LLM-as-judge, and what are its risks?

Using a strong model to score outputs against a rubric. It scales and correlates well with humans when calibrated — but it has biases: it favours longer answers, favours the first option, and can prefer its own style. Mitigate by randomising order, giving a precise rubric, and reporting agreement against a human-labelled sample.

Q. What is a golden dataset, and how do you build one?

A set of representative inputs with known-good outputs or labels, used as the regression suite for the system. Build it from real usage, cover the edge cases and the questions that should be refused, and keep it small but honest — 50 well-chosen cases beat 5,000 generated ones.

Q. What is RAGAS?

An open framework for RAG evaluation that scores things like faithfulness (is the answer supported by retrieved context?), answer relevance and context precision and recall. It is a useful starting point, but treat its scores as a signal, not ground truth — validate against human labels.

Q. How do you prevent regressions when you change a prompt or model?

Run the golden set in CI on every change and fail the build if a key metric drops beyond a threshold. Version prompts and pin model snapshots so a silent provider update cannot change behaviour underneath you. Report the before-and-after numbers in the pull request.

Q. How do you measure quality in production?

Combine offline evals with online signals: thumbs up and down, task completion or deflection rate, and sampled human review. Trace every request so you can pull the exact retrieval and prompt for any bad answer. Watch for quality drift when the provider updates a model.

E. Serving, latency and cost

Q. Your endpoint's p95 latency and cost are both too high — what do you do?

Measure first to find the bottleneck. Then pull levers in impact order: route easy queries to a cheaper model, cache repeats (including semantic caching), trim context (retrieve instead of stuffing), cap and shape output, batch offline work at half price, and use an optimised serving stack. Confirm the bottleneck before optimising anything.

Q. How do you serve an LLM efficiently?

Continuous (in-flight) batching to keep the GPU busy, KV-cache management (paged attention) to cut memory waste, quantisation to fit bigger models, and prefix caching to reuse shared prompts. Serving is memory-bandwidth-bound, so these target memory. Use vLLM or TGI rather than a naive loop.

Q. How do you cut the cost of a high-volume LLM product without hurting quality?

Meter first to find the one or two paths that dominate spend. Then: route and cascade — a cheap model first, escalate on failure; cache prompts and semantic repeats; trim context with RAG rather than stuffing; cap output length, since output is the pricey half; and batch offline jobs. Model choice dominates because tiers differ 10–30x.

Q. What is semantic caching?

A cache keyed on the meaning of a query (its embedding) rather than exact text, so a near-duplicate question returns the stored answer without another LLM call. Powerful and risky: set the similarity threshold too loose and you serve a stale answer to a subtly different question.

Q. How do you reduce hallucination?

Ground the answer with RAG, require citations, and add a refusal path so ‘I do not know’ is allowed when retrieval is empty. Add a faithfulness check between the answer and its sources, lower the temperature for factual tasks, and monitor. You reduce hallucination; you do not eliminate it.

Q. What is model routing?

Sending each request to the cheapest model that can handle it, reserving the expensive flagship for hard cases — often a cascade where a cheap model answers and escalates on low confidence. Because model tiers differ by 10–30x in price, routing is usually the single biggest cost lever in a production system.

F. System design

Q. Design a RAG system for an enterprise knowledge base.

Clarify scope first: corpus size, update frequency, permission model, latency and cost budget. Then: connectors and an ingestion pipeline (parse, structure-aware chunk, enrich metadata, embed, index); a query path of authenticate → permission filter → hybrid retrieval → rerank → assemble a small high-quality context → grounded, cited generation → abstain when nothing scores above a threshold; and a reliability layer of evaluation, tracing, caching and retries. State your trade-offs — chunk size, model tier, context length — and your two biggest risks: faithfulness and freshness.

Q. How would you design an agent that triages support tickets?

Define the task and the definition of done. Give the agent a small tool set: classify, look up the customer and history, search the knowledge base, draft a reply, and — as a gated write action — apply a label or escalate. Allowlist writes, require human approval for anything customer-facing, cap steps, and log every action. Evaluate on a task-success set and review failed traces.

Q. How do you handle a prompt-injection attack through retrieved content?

Treat all retrieved content as untrusted. Label each passage with an id, instruct the model to answer only from the passages and cite them, and verify after generation that every citation maps to a passage that was actually in the prompt — strip or flag any that do not. Add an abstention gate, and never let retrieved text reach a tool that performs a write.

Q. What does the default 2026 production retrieval stack look like?

Structure-aware chunking with overlap; a hosted or self-hosted embedding model used consistently at index and query time; a vector store with an ANN index (HNSW is the common default) plus keyword search; reciprocal rank fusion; a cross-encoder reranker; grounded generation with citations and an abstention path; a golden-set evaluation; and tracing on every request. Treat the LLM call as the easy last step and spend your design time on retrieval quality, grounding and cost.

How to answer any of these well

State the trade-off, name a concrete tool, and mention what you would measure. Interviewers are not looking for the one right answer — they are checking whether you have built the thing and thought about what breaks.

PART 11 Getting Hired — Resume, LinkedIn, GitHub

Your resume, rewritten for AI roles

Recruiters and keyword filters scan for the skills in the postings. Rewrite your bullets to lead with outcomes and the AI skills the role asks for — not the technologies you touched.

Weak bullet vs strong bullet

- **Weak:** “Worked on a chatbot using LangChain and OpenAI API.”
- **Strong:** “Built a RAG service over 12,000 internal documents with hybrid retrieval and reranking; measured recall@5 of 0.86 and faithfulness of 0.94 on a 60-question golden set; cut cost per query 60% with semantic caching.”

The strong bullet wins because it has **scale, a metric and a cost number**. Every project on your resume should carry at least one measured number.

The keyword set to include (only if true)

RAG, retrieval-augmented generation, embeddings, vector database, hybrid search, reranking, LLM API, prompt engineering, tool calling, agents, MCP, evaluation, LLM-as-judge, golden dataset, fine-tuning, LoRA, MLOps, vLLM, observability, Python, FastAPI, Docker, AWS. Mirror the exact wording of the job description where it is honest to do so — the filter is literal.

LinkedIn and GitHub

- **Headline:** “AI Engineer — RAG, agents and evaluation | Python, FastAPI, vector databases”. Put the target role in the headline even during the switch; recruiters search titles.
- **Featured section:** pin your two best project repos and the live demo links. This is the first thing a recruiter clicks.
- **About:** three sentences — what you build, the metrics you can show, and what you are looking for.
- **GitHub profile README:** a short pitch, your two or three pinned projects with one-line metrics, and links. Recruiters often read the profile before the repo.
- **Activity:** post one short technical note a week — a failure you debugged, a benchmark you ran. Visible, specific work gets inbound interest.

The five-minute reviewer test

Before you apply anywhere, open each project repo as a stranger would. Can you see, without running anything, what it does, that it works (a number), and how it is built (a diagram)? If not, fix the README first. This single habit converts more applications into interviews than any extra course.

Apply before you feel ready

Start applying in week 9 of the roadmap, with two projects done. The market gives you signal — which roles reply, what they ask — that no amount of extra study gives you. Treat the first ten applications as market research, not a verdict on you.

Salary negotiation in India — the short version

-
- **Never give the first number** if you can avoid it. “I would rather understand the role and the band for this level first” is a complete answer.
 - **Research the band** using the ranges in Part 1 and by asking peers in the same role and city. Know your number before the call, and a walk-away number below it.
 - **Negotiate the whole package:** fixed pay, variable and bonus, joining bonus, ESOPs, the level/title, and remote or hybrid terms. For a career switch, the **level** often matters more than a few percent of fixed pay, because it sets your trajectory.
 - **Use competing offers** — honestly and specifically. “I have another offer at X; I would prefer to join you — can you get closer?” works far more often than a bluff.
 - **Get it in writing** before you resign. Nothing is real until it is in the offer letter.

This is general market guidance, not financial or legal advice. Offers and tax treatment vary by employer and city.

PART 13 Your 30-Day Action Plan

If you do nothing else with this kit, do this. Thirty days, three goals, one checklist each. Tick them as you go.

Days 1-10 — Foundations and the first artefact

- ☐ Call a model API directly from a script and stream the response.
- ☐ Build a small app that returns validated structured JSON.
- ☐ Set up your GitHub profile README with a one-line pitch.
- ☐ Write your first public technical note (200–400 words).

Days 11-20 — Your first real project

- ☐ Build the RAG service (Project 1) over a real document set.
- ☐ Build a golden set of 50+ questions with labelled source chunks.
- ☐ Report recall@5, faithfulness, cost per query and p95 latency in the README.
- ☐ Add an architecture diagram and a trade-offs section.

Days 21-30 — The differentiator and the applications

- ☐ Build the eval harness (Project 3) and wire it into CI.
- ☐ Rewrite your resume bullets with metrics and the keyword set from Part 11.
- ☐ Rehearse the ten most likely interview questions from Part 10 out loud.
- ☐ Send your first ten applications, with a tailored line for each role.

The one habit that changes the outcome

Ship something public every week. One commit, one note, one demo. The compounding of visible work is what turns a quiet GitHub into inbound interviews — and it is the single thing most candidates never do.

Glossary — the terms you must be able to define

Term	Plain-English meaning
LLM	Large language model — a model that predicts the next token to generate text
Token	A piece of text, roughly three-quarters of a word; the unit providers bill
Context window	The maximum tokens a model can consider at once
Embedding	A vector representation of text where similar meanings are close together
Vector database	A store built for fast similarity search over embeddings
Chunking	Splitting documents into retrieval-sized pieces
Hybrid search	Combining keyword (BM25) and vector search
Reranker	A cross-encoder that re-orders retrieved candidates by true relevance
RAG	Retrieval-augmented generation — answering from retrieved documents
Agent	A model in a loop that calls tools and decides the next step

Term	Plain-English meaning
Tool / function calling	Letting the model request a function your code then runs
MCP	Model Context Protocol — a standard for connecting models to tools and data
Evaluation (evals)	Automated measurement of output quality against a golden set
LLM-as-judge	Using a strong model to score outputs against a rubric
Golden dataset	A set of inputs with known-good answers used as a regression suite
LoRA / QLoRA	Parameter-efficient ways to fine-tune a model cheaply
vLLM	A high-throughput engine for serving LLMs with batching
Semantic caching	Caching answers by the meaning of a query, not exact text
Prompt injection	Malicious instructions hidden in content a model reads

Sources and verification. Market and hiring figures are drawn from NASSCOM's 2026 AI talent analysis, Naukri JobSpeak (September 2026), Xpheno's Active Tech Jobs Outlook (October 2026), CIEL HR agentic-AI demand analysis, and India Today and Business Today reporting of the same period. Salary bands follow 2026 industry compensation reporting. Model names and prices are from the providers' own pricing pages and public launch posts, verified October 2026. Educational guidance only; not legal, financial or immigration advice.

Thank you for reading. If this kit helped you, tell us what worked — and what we should add next. © 2026 Digitalaikart · lonefaisal977@gmail.com